

Pag-connect ng Frontend sa FastAPI Backend

Step-by-step guide: CORS, Login, Token Storage, at Role-Based Authorization
eSaka Project

HTML5 · CSS3 · Vanilla JS · FastAPI · PostgreSQL

Buod ng Buong Proseso

Ang pag-connect ng frontend (HTML/CSS/JS) sa FastAPI backend ay binubuo ng **5 pangunahing hakbang**: CORS setup, password hashing, login request, token storage, at role-based authorization. Sinusundan ng bawat hakbang ang nauna — huwag laktawan.

#	Hakbang	Nasaan
1	I-enable ang CORS	Backend (FastAPI)
2	I-hash ang passwords	Backend (FastAPI)
3	Gumawa ng /login endpoint na nagbabalik ng JWT token	Backend (FastAPI)
4	I-connect ang login form gamit ang fetch()	Frontend (login.html)
5	I-protektahan ang bawat dashboard endpoint base sa role	Backend (FastAPI)

1 I-enable ang CORS sa Backend

Dahil magkaiba ang port ng Live Server (karaniwang 5500) at FastAPI (8000), ituturing ito ng browser na "ibang address" — kailangan ng explicit na pahintulot mula sa backend.

```
# src/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://127.0.0.1:5500",
        "http://localhost:5500",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

⚠ **Mahalagang Paalala**
Huwag gamitin ang allow_origins=["*"] (wildcard) kasabay ng allow_credentials=True — invalid na combination ito sa CORS spec at hindi ito gagana. Palaging mag-list ng exact na origins.

2 I-hash ang mga Password (hindi Encrypt)

Hashing (one-way, hindi na mababalik) ang tamang gamitin para sa passwords — hindi **encryption** (reversible, may key). Gamitin ang bcrypt sa pamamagitan ng passlib .

```
pip install passlib[bcrypt]

from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Paggawa ng account (System Admin side):
hashed_password = pwd_context.hash(plain_password) # ito ang i-SAVE sa database

# Pag-verify (sa login):
is_valid = pwd_context.verify(plain_password, hashed_password) # True / False
```

3 Gumawa ng /login Endpoint (JWT Token)

```
pip install python-jose[cryptography]
```

```
from datetime import datetime, timedelta
from jose import jwt
from pydantic import BaseModel
from fastapi import HTTPException

SECRET_KEY = "palitan-mo-ito-ng-sarili-mong-secret-key"
ALGORITHM = "HS256"

class LoginRequest(BaseModel):
    username: str
    password: str

def create_access_token(data: dict, expires_minutes: int = 60):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=expires_minutes)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

@app.post("/api/auth/login")
def login(data: LoginRequest):
    user = get_user_from_db(data.username) # sarili mong DB query

    if not user or not pwd_context.verify(data.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    token = create_access_token({"sub": user.username, "role": user.role})

    return {
        "access_token": token,
        "role": user.role,
        "user_id": user.id
    }
```

4 I-connect ang Login Form (Frontend)

Sa `login.html`, palitan ang placeholder/console.log ng totoong `fetch()` call:

```
const loginForm = document.querySelector("#loginForm");

loginForm.addEventListener("submit", async (e) => {
  e.preventDefault();

  const username = document.querySelector("#username").value;
  const password = document.querySelector("#password").value;

  try {
    const response = await fetch("http://127.0.0.1:8000/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ username, password }),
    });

    const data = await response.json();

    if (!response.ok) {
      alert("Login Failed: " + (data.detail || "Invalid credentials"));
      return;
    }

    // I-save ang token at role sa browser
    localStorage.setItem("token", data.access_token);
    localStorage.setItem("role", data.role);
    localStorage.setItem("userId", data.user_id);

    redirectUserByRole(data.role);

  } catch (error) {
    console.error("Network error:", error);
    alert("Hindi maabot ang server. Siguraduhing running ang backend.");
  }
});

function redirectUserByRole(role) {
  if (role === "DA") window.location.href = "dashboards/da.html";
  else if (role === "PROVINCIAL") window.location.href = "dashboards/provincial.html";
  else if (role === "MUNICIPAL") window.location.href = "dashboards/municipal.html";
  else if (role === "AEW") window.location.href = "dashboards/aew.html";
  else window.location.href = "dashboards/system-admin.html";
}
```

💡 Tip

I-verify muna gamit ang DevTools Console (F12) bago mo i-wire sa actual form: `fetch('http://127.0.0.1:8000/').then(r=>r.json()).then(d=>console.log(d))`. Kung 200 ang balik, gumagana ang koneksyon.

5 Gamitin ang Token sa Dashboard Pages

Sa loob ng `dashboards/da.html` (o kahit anong dashboard), basahin ang naka-save na token para makakuha ng protected data:

```
document.addEventListener("DOMContentLoaded", async () => {
  const token = localStorage.getItem("token");

  if (!token) {
    alert("Unauthorized access. Please login.");
    window.location.href = "../login.html";
    return;
  }

  try {
    const response = await fetch("http://127.0.0.1:8000/api/da/reports", {
      method: "GET",
      headers: {
        Authorization: `Bearer ${token}`,
        "Content-Type": "application/json",
      },
    });

    if (response.status === 401) {
      localStorage.clear();
      window.location.href = "../login.html";
      return;
    }

    const reportData = await response.json();
    console.log("DA Data:", reportData);
    // Dito mo na i-render ang data sa mga tables/charts ng UI

  } catch (err) {
    console.error("Error loading dashboard data:", err);
  }
});
```

6 Kritikal: I-enforce ang Role sa Backend Mismo

🚨 Ito ang pinaka-madalas makaligtaan

Ang `if (!token)` check sa Step 5 ay **UX lang, hindi security**. Kung basta valid token lang ang tinitignan ng backend endpoint (kahit kaninong role), pwedeng gamitin ng isang FARMER ang sarili niyang valid token para tumawag sa `/api/da/reports` at makakuha pa rin ng datos.

Kailangan i-check ang role sa loob mismo ng bawat protected endpoint:

```
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="api/auth/login")

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return {"username": payload.get("sub"), "role": payload.get("role")}
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid or expired token")

def require_role(required_role: str):
    def checker(current_user = Depends(get_current_user)):
        if current_user["role"] != required_role:
            raise HTTPException(status_code=403, detail="Not authorized for this role")
        return current_user
    return checker

@app.get("/api/da/reports")
def get_da_reports(current_user = Depends(require_role("DA"))):
    # dito lang makakapasok kung "DA" talaga ang role sa loob ng VERIFIED token
    return {"reports": [...]}
```

Bakit importante ito: kahit ma-access ng maling user ang `da.html` file mismo (walang pumipigil dun, HTML file lang yun), walang mangyayari kasi laging 403 (Not Authorized) ang isasagot ng backend sa kanya — walang datos na lalabas sa page.

7 Logout Flow

```
function handleLogout() {
    localStorage.clear(); // o localStorage.removeItem() per key kung may iba pang laman
    window.location.href = "../login.html";
}
```

Common Errors & Ayos

Error sa Console	Ibig Sabihin	Ayos
"blocked by CORS policy"	Hindi tugma ang origin ng frontend sa allow_origins ng backend	I-check ang exact URL sa address bar (127.0.0.1 vs localhost), idagdag pareho sa allow_origins
"Failed to fetch"	Hindi tumatakbo ang backend, o maling URL/port	I-verify na naka-run ang <code>uvicorn main:app --reload</code>
401 Unauthorized	Walang token, expired na, o maling credentials	I-check kung na-send ang Authorization header nang tama
403 Forbidden	Valid ang token pero maling role para sa endpoint na ito	Tama ito — gumagana ang role check mo!

Buod ng Checklist

Parte	Status
CORS na may specific origins (hindi wildcard)	Gawin
Password hashing gamit ang bcrypt	Gawin
/login endpoint na nagbabalik ng JWT + role	Gawin
Frontend fetch() papalit sa hardcoded check	Gawin
Token storage sa localStorage	Gawin
Role-based redirect (UX)	Gawin
Role-based authorization sa BACKEND endpoints	Kritikal — huwag kalimutan
Logout na nag-cclear ng token	Gawin